



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FUNDAMENTALS OF OPTIMIZATION

Metaheuristic methods

ONE LOVE. ONE FUTURE.

CONTENT

- Introduction to Metaheuristics
- Tabu Search (TS)
- Simulated Annealing (SA)
- Genetic algorithms (GA)
- Ant Colony Optimization (ACO)

- Metaheuristics
 - Generic framework, problem-independent
 - Aim to find high-quality solutions to optimization problems in reasonable time
 - Do not specify (like heuristic methods) how close are the obtained solution from the optimal solutions
 - Single-solution based: Tabu Search, Simulated Annealing, Variable Neighborhood Search, etc.
 - Population-based: Genetic algorithms, Ant Colony Optimization, etc.
- Combinatorial Optimization Problems
 - Without loss of generality, consider a quality function f to be minimized
 - Quality function f can be
 - Objective function in pure optimization problems (no constraints)
 - Combination of the objective function and constraint violations in constrained optimization problems

Tabu Search

- Based on neighborhood search
- Use memory structure to avoid revisiting solutions explored so far

```
function TabuSelect( $N$ ,  $f$ ,  $s$ ,  $T$ ){  
    eval =  $\infty$ ;  $s$  = NULL;  
    for  $x$  in  $N(s)$  do  
        if  $x \notin T$  and  $f(x) < \text{eval}$  then {  
            eval =  $f(x)$ ;  $s$  =  $x$ ;  
        }  
    return  $s$ ;  
}
```

```
function TabuSearch( $N$ ,  $f$ ){  
    //  $N$ : neighborhood  
    //  $f$ : quality function  
     $s$  = Generate an initial solution;  
     $s^*$  =  $s$ ;  
     $T$  = initialize a tabu list  
    while termination not reach do {  
         $s$  = TabuSelect( $N(s)$ ,  $f$ ,  $s$ ,  $T$ ); //neighbor selection  
        if  $s$  = NULL then break;  
        if  $f(s) < f(s^*)$  then     $s^*$  =  $s$ ;  
        AddLast( $s$ ,  $T$ ); // add  $s$  to the end of  $T$   
    }  
    return  $s^*$ ;  
}
```

Tabu Search – implementation issues

- Storing all solutions visited so far and check if a solution is in the tabu list is not efficient
- In practice
 - Store the move that generates the new solution instead of storing the new solution
 - Maintain the tabu list with a limited length (tabu length)
 - Aspiration criterion
 - Accept the move that is in the tabu list but it generates a solution that is better than the best solution found so far

Simulated Annealing

- Probabilistic technique based on neighborhood search
- Accept poor solutions with a probability which reduces over time

```
function S-Metropolis( $N, f, s, t$ ){  
    select  $n \in N$  with probability  $1/|N|$ ;  
    if  $f(n) < f(s)$  then return  $n$ ;  
    else with probability  $e^{\frac{f(s)-f(n)}{t}}$ ;  
    else return  $s$ ;  
}
```

```
function SimulatedAnnealing( $N, f$ ){  
    //  $N$ : neighborhood  
    //  $f$ : quality function  
     $s$  = Generate an initial solution;  
     $s^* = s$ ;  
     $t$  = initial temperature;  
    while termination not reach do {  
         $s = \text{S-Metropolis}(N(s), f, s, t)$ ; //neighbor selection  
        if  $s = \text{NULL}$  then break;  
        if  $f(s) < f(s^*)$  then  $s^* = s$ ;  
         $t = \text{update}(t)$ ; // reduce temperature  
    }  
    return  $s^*$ ;  
}
```


Genetic algorithms

- Population-based method
- Generate initial population of solutions
- Perform cross over between 2 parent solutions from the population for generating new child solutions
- Perform mutation on child solutions with some probability
- Replace some poor solutions in the population by new good child solutions generated

Genetic algorithms

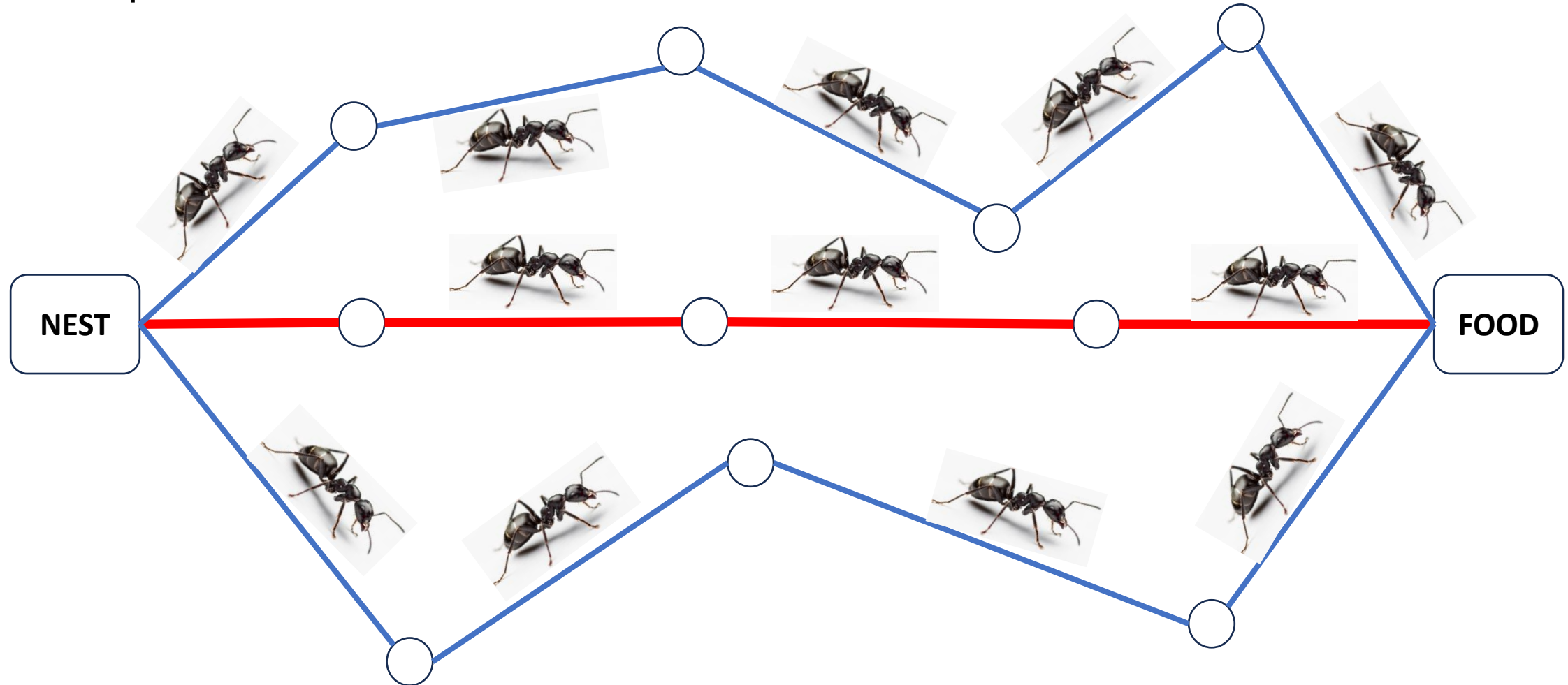
```
GA(population_size, generations, mutation_rate, replacement_rate, crossover_rate){
    population = Generate an initial population;
    s* = best(population);
    for g = 1 to generations do {
        s' = best(population);
        if f(s') < f(s*) then s* = s';
        new_population = {};
        for i = 1 to population_size*crossover_rate do {
            P1, P2 = SelectParent(population);
            C1, C2 = CrossOver(P1,P2);
            Mutate(C1, mutation_rate); Mutate(C2, mutation_rate);
            new_population.add(C1, C2);
        }
        population = UpdatePopulation(population, new_population, replacement_rate);
    }
    return s*;
}
```

Genetic algorithms

```
SelectParent(population){  
  for  $I \in \text{population}$  do  $f(I) = \text{Fitness}(I)$ ;  
   $S = \sum_{I \in \text{population}} f(I)$ ;  
  for  $I \in \text{population}$  do  $p(I) = \frac{f(I)}{S}$ ;  
   $r = \text{random}(0,1)$ ; // random value from 0 to 1  
   $A = 0$ ;  
  for  $I \in \text{population}$  do {  
     $A = A + p(I)$ ;  
    if  $A \geq r$  then return  $I$ ;  
  }  
}
```

Ant Colony Optimization (ACO)

- Nature-inspired metaheuristic



Ant Colony Optimization (ACO)

- Nature-inspired metaheuristic
- Each iteration
 - m ants construct m solutions exploiting a pheromone model
 - Solution can be seen a sequence of components: component (i, j) means (for instance) the assignment of the decision variable x_i by the value j
 - Select component (i, j) with the probability

$$p(i, j) = \frac{\tau(i, j)^{\alpha} \eta(i, j)^{\beta}}{\sum_{(p, q) \in \text{Candidate}} \tau(p, q)^{\alpha} \eta(p, q)^{\beta}}$$

$\tau(i, j)$: pheromone on component (i, j)

$\eta(i, j)$: heuristic function for selecting component (i, j)

Ant Colony Optimization (ACO)

- Some obtained solutions will be used to update pheromone

- Example: $\tau(i, j) = (1 - \rho) \tau(i, j) + \sum_{k=1}^m \Delta(i, j)^k$

in which

- $0 < \rho \leq 1$: pheromone evaporation rate
 - $\Delta(i, j)^k = \begin{cases} \frac{1}{C(k)}, & \text{if } (i, j) \text{ is selected in the solution constructed by ant } k \\ 0, & \text{otherwise} \end{cases}$

$C(k)$: the cost of the solution constructed by ant k (in the minimization problem)

Ant Colony Optimization (ACO)

```
ACO(m, generations,  $\alpha$ ,  $\beta$ ,  $\rho$ ){
   $s^* = \perp$ ; Cand = candidates of components for selection;
   $\tau(i,j)$  = initial values,  $\forall (i,j) \in \text{Cand}$ ;
  for g = 1 to generations do {
    for k = 1 to m do {
      s[k] = ConstructSolution[k](Cand,  $\tau$ ,  $\alpha$ ,  $\beta$ );
      if s[k] is better then  $s^*$  then  $s^* = s[k]$ ;
    }
    for k = 1 to m do {
      if (i,j) is in s[k] then  $\Delta(i,j)^k = 1/f(s[k])$ ; else  $\Delta(i,j)^k = 0$ ;
    }
    for (i,j) in Cand do {  $\tau(i,j) = (1-\rho) * \tau(i,j) + \sum_{k=1}^m \Delta(i,j)^k$ ; }
  }
  return  $s^*$ ;
}
```

```
ConstructSolution[k](Cand,  $\tau$ ,  $\alpha$ ,  $\beta$ ){
  s = {}; // initialize an empty solution
  C = Cand;
  while not solution(s) and C  $\neq \emptyset$  do {
    for (i,j) in C do  $p(i,j) = \frac{\tau(i,j)^\alpha * \eta(i,j)^\beta}{\sum_{(p,q) \in C} \tau(p,q)^\alpha * \eta(p,q)^\beta}$ 
    (i,j) = SelectWithProbability(C, p);
    s = s  $\cup \{(i,j)\}$ ; C = Update(C, (i,j));
  }
  return s;
}
```

```
SelectWithProbability(Cand, p){
  x = random(0,1); s = 0;
  for e  $\in$  Cand do {
    s = s + p(e); if s  $\geq$  x then return e;
  }
}
```

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a white, bold, sans-serif font.

HUST

THANK YOU !